Serial Protocol Decode — DMM6500

A UART decoder that runs **on** a Keithley DMM6500 bench multimeter. It digitizes the line with the instrument's own digitizer, recovers the baud rate, frame format and idle polarity from the signal, and shows the bytes on the front panel as text or hex. No host, no logic analyser, one probe.

Ian Ameline · **version 1.04 — beta** · MIT licence (see LICENSE) · user manual

Beta status. Everything the manual claims was measured on the bench, and the release gate below is what measures it — it must pass with zero failures before a build goes out. One rough edge worth naming here: **flow control** is verified electrically — 4.92 V, ~10 µs, one credit pulse per armed capture — but has never been closed as a loop against a device that actually waits for it, so the "unlimited with flow control" claim is sound by construction rather than demonstrated. The pulse width is **not settable** on this firmware — `trigger.extout.pulsewidth` is nil on 1.7.17a — and the default measures **9.4–9.8 µs**, so a device waiting for a credit needs an edge-triggered input rather than a polling loop.

Automatic rate detection has two known failures, both rare and both fixable by typing the rate in — which is what most decoders require of you anyway. A short pattern repeated over and over — `00/FF/55/AA`, walking bits, a fixed string on a loop — can be measured at a small multiple of its true rate: **24 of 6,714** bench points across four full sweeps, **0.36 %**, every one a repeating pattern at a non-standard rate. Offline the split is stark — **0 of 6,314 standard-rate points** against 24 of 6,027 non-standard ones — but on the instrument a waveform carrying a **LIN-style break field**, which is not valid 8N1 at all, *was* measured at three times its true rate on standard rates, silently. No ordinary UART payload has shown it at any standard rate. Separately, a line whose logic low sits around **−2 V** can misdetect, while −0.1, −0.5, −1.0 and −3.0 V all measure clean. Both are characterised with their numbers under **Known failures of automatic rate detection** in the manual.

Endurance, measured. A 7-hour soak of this build on one power cycle ran **81 laps — 3,726 capture-and-decode cycles and 8 one-press recordings — with zero failures.** Every recording ended because its buffer filled, at 8 192 bytes, with no instrument events logged and no error left behind. The number worth more than the pass count is the **lap time**: a mean of 312.8 s over the first ten laps and 315.7 s over the last ten, **+0.9 %** across the full seven hours, so nothing accumulated. A leaked display object, a buffer handle never returned or a fragmenting allocator would each show up as laps getting slower — which matters because a leak here would not fail a lap, it would wedge the instrument around hour six.

What that soak does **not** cover: it is depth rather than breadth — 43 stimulus points repeated 98 times, no front-panel interaction (that is `hw-panel`'s job), and one power cycle rather than many.

Nothing stops a long job early — there is no working stop control. A touch press cannot be delivered while a script runs, so the panel cannot offer a stop control, and the front-panel TRIGGER key does not fill the gap: it does **not** deliver `trigger.EVENT_DISPLAY` while a panel-initiated run is executing. So a recording runs for its stated time and cannot be interrupted; decide the size before pressing. The user manual says so under **The TRIGGER key**. The same key does still *gate* an armed capture when `Options ▸ Trigger = Trigger key`, which took its own purpose-

built test (`tools/bench_trigkey.py --quiet-line`) because timing-based tests could not tell a prompt press from a free-running capture.

How much it captures, and how many presses. A screenful is ~240 bytes; a recording holds **8 192 or 32 768 bytes** to a file, and **one press** records it, decodes all of it and files it — no stepping. Beyond one window, flow control makes the window a chunk size rather than a total: one press then credits the device, records, decodes and credits again until it stops sending, so an unlimited amount can be captured losslessly **with no interaction per window** by a device built to wait for the credit — bounded at 32 windows or 20 minutes per press, then Mode and Capture resume the same conversation in the next file – the sender is still waiting for its credit, so nothing is lost in the gap. There is deliberately no uncapped mode: continuous decode-to-file is drain-bound at 2400 baud, too slow to be worth a mode. The arithmetic is in docs/REFERENCE.md.

Ships as `Serial_Decode.tspa`, installed from a USB key through the instrument's own **Manage Apps** screen. Written in TSP — Lua **5.0.2** embedded in the instrument firmware, which is why the sources avoid `#`, `%`, `string.gmatch` and bitwise operators throughout.

Before releasing it to anyone

Three gates, each about ten times the cost of the one before it. Run them in order, so a cheap failure stops an expensive one. `docs/BENCH.md` describes the harness in full.

```
python3 tools/release_sweep.py --offline   #  ~1 min, no instruments
python3 tools/bench_smoke.py               #  11 min, both instruments
python3 tools/soak.py --hours 8 --suites formats,plan --skip-vectors v95,v96
```

The smoke gate is 11 minutes and covers the whole rate range. Four waveforms — the fox and a random payload, each in 8N1 and 7E1 — paired crosswise so one pair takes the 22 standard baud rates and the other the 21 drawn ones, and each rate family therefore faces both formats and both content classes. Then all 45 button presses. 86 cells, 9.5 min; presses, 1.9 min. Run it after any change and before starting anything long: a soak lap is three hours, and a harness defect found at minute 147 has cost an evening.

No version is tagged without at least 8 hours of soak. Not a guideline. A release sweep answers "does it pass?" once, which is the wrong question for anything that fails one run in ten — a single green sweep licenses an intermittent and a single red one reads as a regression nobody can reproduce. The soak runs the seeded sweep for hours and reports a **failure rate per test point**, which is the number an intermittent actually has. At a measured 6.5 s per cell a lap of 39 waveforms across 43 baud rates is about three hours, so eight hours is between two and three laps – the least that can distinguish "fails every lap" from "failed once".

Code changes do not get pushed without passing the smoke gate, and that is enforced rather than remembered. On a pass, `bench_smoke.py` writes a receipt holding a hash of `tsp/` and `tools/` as they were; a `pre-push` hook recomputes it and refuses if either tree has moved since. So the one-line fix made *after* the run invalidates the receipt, which is the case worth catching — that change is the least tested thing in the push. Install it with:

```
ln -sf ../../tools/hooks/pre-push .git/hooks/pre-push
```

Documentation-only pushes are unaffected; neither tree is hashed. When the bench is genuinely unavailable — mid-soak, say — `SMOKE_OVERRIDE="reason" git push` proceeds and prints the reason, so the exception is a decision on record rather than a silent bypass.

Every lap draws a different waveform order, different non-standard rates and a different wait before each capture, all from the lap number — so `--iteration N` rebuilds any lap exactly, without running the ones before it, and a failure replays offline in seconds. `docs/BENCH.md` has the commands.

```
python3 tools/release_sweep.py
```

That is the gate. It runs every check in one go and writes an auditable record to `out/release/<timestamp>/` — a `REPORT.md`, a `summary.json`, the full output of every stage, every front-panel screenshot taken, and the exact `.tspa` and `MANUAL.pdf` the results describe with their SHA-256s. It exits non-zero if any gate fails.

It needs a freshly power-cycled DMM6500. `sdec.start()` builds the display objects and the firmware does not fully return the pool, so the app refuses a second build in one power cycle. The sweep checks this up front and refuses with the remedy rather than failing forty minutes in. It also needs the SDG2122X generator with the stimulus waveforms loaded (`bench_matrix.py --upload` puts them there) and only one client may hold the DMM's control socket at a time.

For a code change, the offline half runs in under ten seconds and needs no instruments:

```
python3 tools/release_sweep.py --offline
```

Stage	Checks
`lint parse`	Lua 5.0.2 incompatibilities; `luac -p` on every module
`vecrefs`	every waveform id named in `tools/` is in `MAP` or declared `RETIRED` — a deletion from `MAP` alone leaves references that fail where they are used, not where they are declared
`soakrand`	`mt19937.py` and `mt19937.lua` produce one sequence, checked against CPython's `random`, which is the same algorithm — plus the 5.0.2 syntax scan that decides whether the module can load on the instrument at all
`unit`	1,047 offline tests — decoder, UI, state machine, file paths, every visible ASCII glyph
`unit-cancel`	one-press recordings, both window sizes, and the flow-control loop running with no interaction
`stress`	hostile signals: never silently **wrong**, never **raises**
`unit-analog`	the bench cases at the app's own sample rates, swept over sampling phase, jitter, noise and where the capture window opens
`unit-phasesweep`	every stimulus vector × capture start × phase/jitter/noise, sharded across the cores — no raise, no result without a format, no wrong byte among those ERR calls trustworthy
`unit-seam`	a capture whose arb loop seam lands late must still be judged, and the narrower trim must still be required
`unit-sdgguard`	every route by which an out-of-spec waveform could reach the generator refuses it
`unit-loremgate`	the harness's own long-payload verdict — every clean run validated, flag count bounded
`tolerance`	recomputes the envelope table printed in the manual
`package archive`	rebuilds the `.tspa`, then builds both screens *from the archive* against a mock front end
`manual`	rebuilds every shipped PDF: [illegible] ceiling, [illegible] DC offsets. A refusal with a reason passes; confident garbage does not

`hw-panel` is the one worth understanding: it checks six things per press — that the handler does not raise, logs no instrument event, returns inside its latency budget, reports what it did, changed the state it was supposed to, *and that the panel actually shows it.* The last is a separate failure from the one before it, because the UI caches its writes: an app can be right while the operator reads something stale.

Layout

`tsp/`	the app. `serial_core` acquisition, `uart_decode` framing, `chunk_decode` resumable decode, `serial_ui` panel, `serial_app` orchestration
`tools/`	harnesses. `release_sweep.py` is the entry point; the rest are the individual authorities it calls
`docs/`	the manual, instrument references, panel mockups
`notes/`	bring-up log, findings, handoffs

`tsp/midi_decode.tsp` and `tsp/lin_decode.tsp` are complete and tested but **not shipped** in version 1 — the LIN checksum has never been checked against a real frame. Re-adding either is one line in `tools/package_tspa.py`; the app discovers what it has at runtime.

Bench

Three instruments over LAN: the DMM6500 under test, an SDG2122X generating the stimulus, an SDS1204X-E for verifying the stimulus is what it claims to be. `tools/instruments.py` holds the addresses and the hazards worth knowing — repeated large waveform uploads wedge the generator's LAN service, and calibration state is off limits on both.